26-2 유니티 스터디

1 주차

정혜원

오늘 만드는 것 & 진행 순서

물리엔진 기반 캐주얼 퍼즐 게임 밑바탕 만들기

1. 안드로이드 세팅 확인
2. 에셋 다운로드 & 임포트
3. 메인 카메라 & 해상도 세팅
4. Playground(바닥·Wall) 제작
5. Dongle(공) 만들고 물리 테스트
6. 마우스로 Dongle 조작하는 로직
7. Screen ↔ World 좌표계 변환

사전 점검 ① 안드로이드 세팅 확인

Edit → Preferences(환경설정) → External Tools
메뉴 맨 아래 Android 세팅 확인

JDK · Android SDK · NDK · Gradle
이 4가지 항목에 노란색 느낌표가
하나도 없어야 정상입니다!

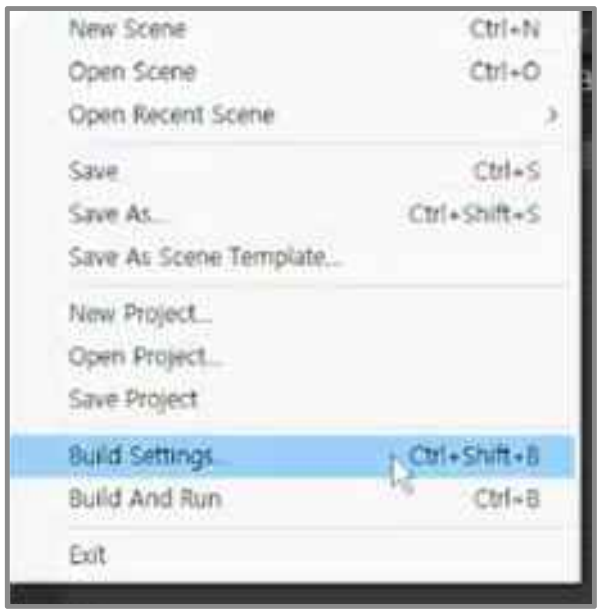
사전 점검 ② 느낌표가 뜬다면

Unity Hub → 설치 탭 →
지금 프로젝트를 열었던 버전 옆 : 클릭
→ 모듈 추가

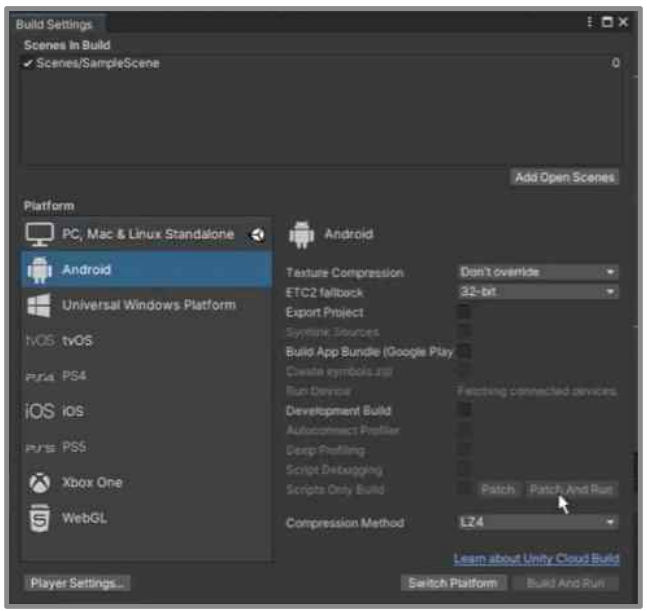
Android Build Support
펼쳐서 SDK & NDK Tools, OpenJDK까지
3개 모두 체크 후 설치
→ 느낌표가 사라지면 완료!

빌드 플랫폼을 안드로이드로

① File → Build Settings



② Android → Switch Platform



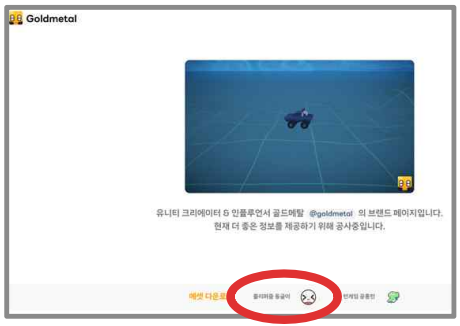
유니티 아이콘이 Android 탭으로 옮겨지면 **완료!**

에셋 다운로드하기

① 구글에 '골드메탈' 검색 → 맨 위 홈페이지 클릭



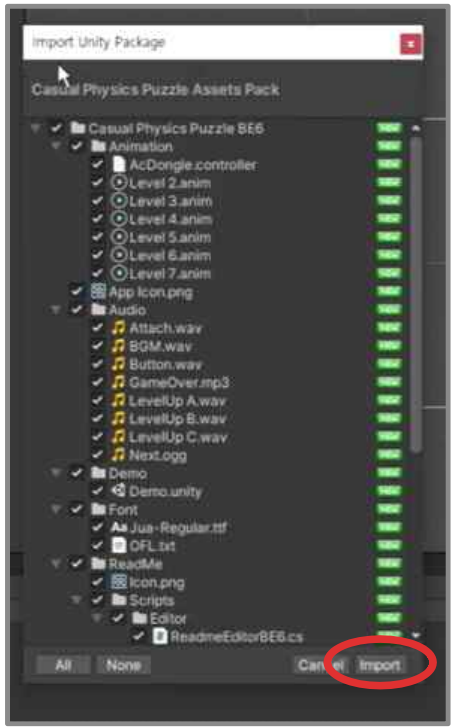
② 홈페이지 하단 '물리퍼즐 동글이' 클릭 → 다운로드



에셋 임포트 & 데모 폴더

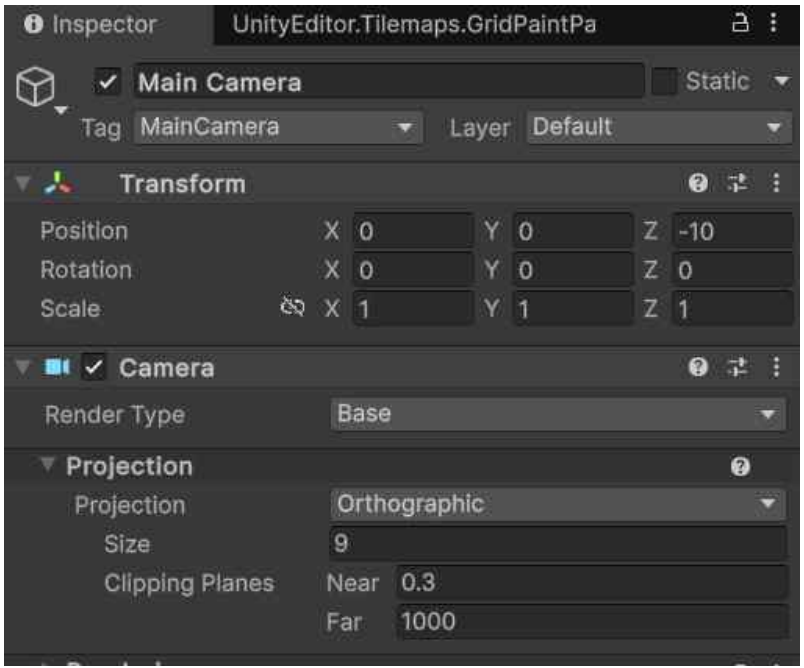
다운받은 파일 더블클릭
→ Import Unity Package 창에서 Import 클릭
→ Inspector에 정보 표시되면 압축 해제 완료

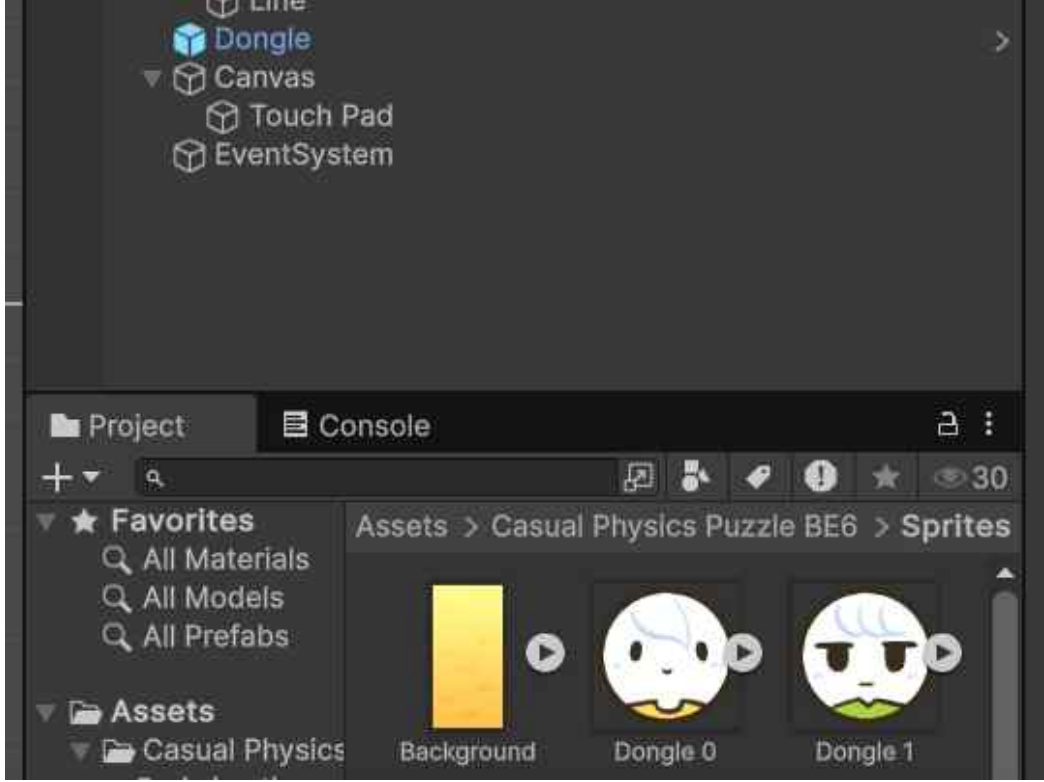
Demo 폴더: 배경이 세팅된 씬 (참고용)
오늘은 원래 샘플 씬에서
처음부터 수동으로 직접 제작합니다!



메인 카메라 세팅

Background 색상 자유 지정 (예시: 흰색)
Size 값을 9로 변경 (필수!)
배경 스프라이트를 하이어라키로 드래그





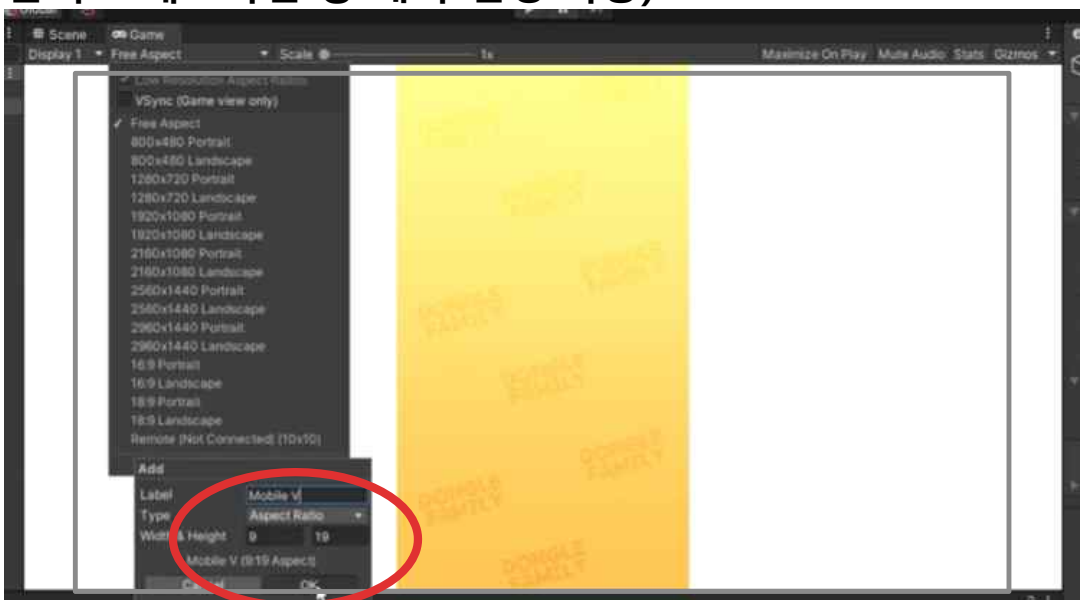
해상도 설정 (세로형 모바일)

세로형 모바일 게임이라 양옆 공간이 불필요

Game 창 → Free Aspect → + → Aspect Ratio → 9:19 추가

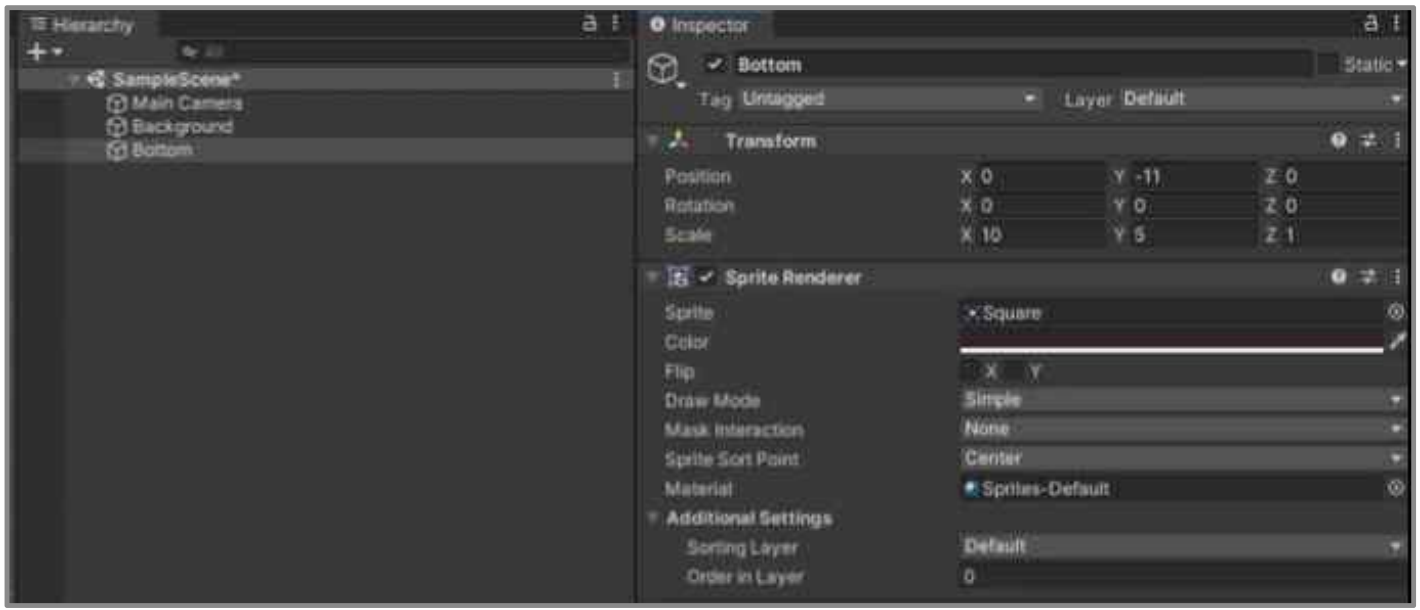
Scene에서 카메라 비율도 자동으로 조정됩니다!

(팁: 에디터 탭을 꾹 눌러 드래그하면 창 배치 변경 가능)



Playground ① 바닥 만들기

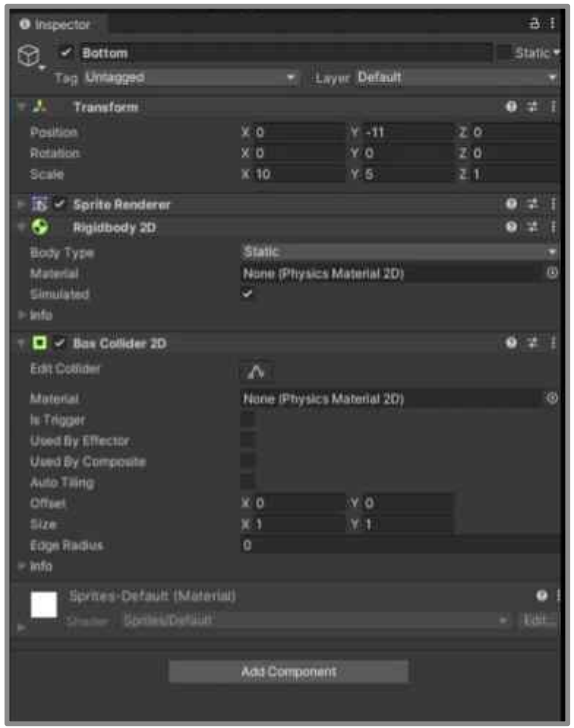
Hierarchy 우클릭 → 2D Object → Sprite → Square
Order in Layer: 배경을 -1로 변경
바닥(Bottom) 완성 값: Scale X10/Y5, Position Y -11



Playground ② 바닥에 물리 적용

Add Component →
Rigidbody2D
바닥은 움직이면 안 되니 Body Type을
Dynamic → Static으로 변경

Box Collider2D 추가
크기는 자동으로 맞춰지므로
손댈 필요 없음 — 바닥 완성!



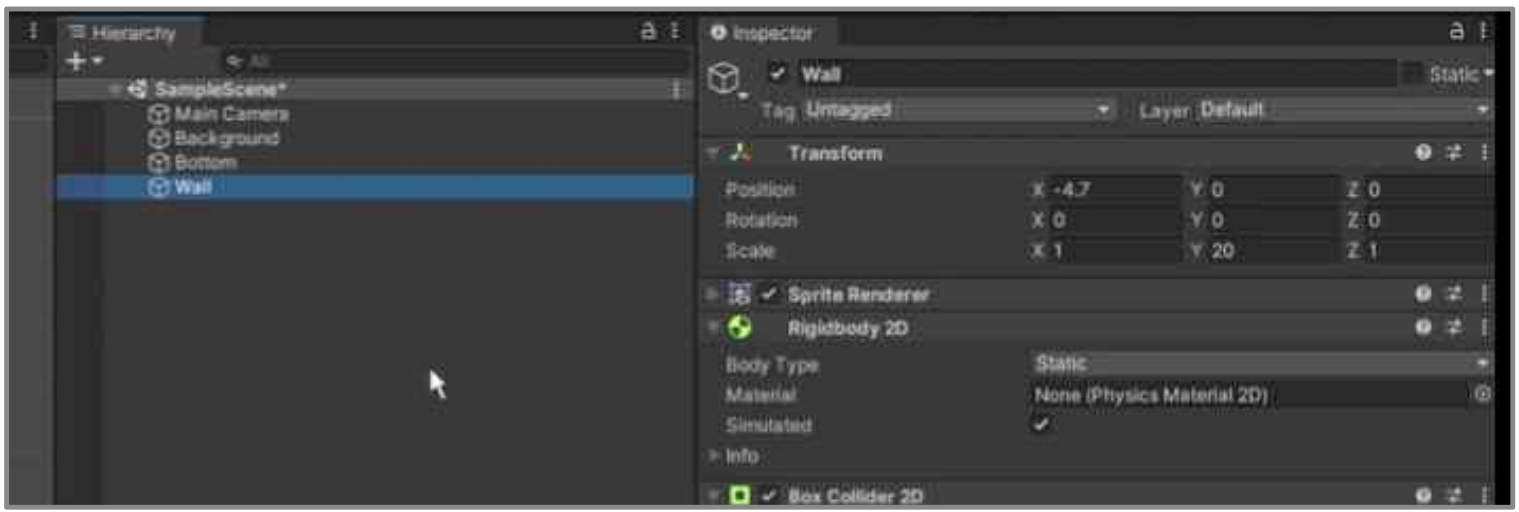
Playground ③ Wall 만들기 (Ctrl+D)

바닥을 Ctrl+D로 복제 → 이름 'Wall', Scale X1/Y20, 위치 Y축0

X축 이동: 카메라에 살짝 닿게 (약 4.7)

다시 Ctrl+D로 복제 → 반대편은 X 부호만 반대로!

왼쪽 Wall (X -4.7) — 오른쪽 Wall은 X 4.7로 대칭

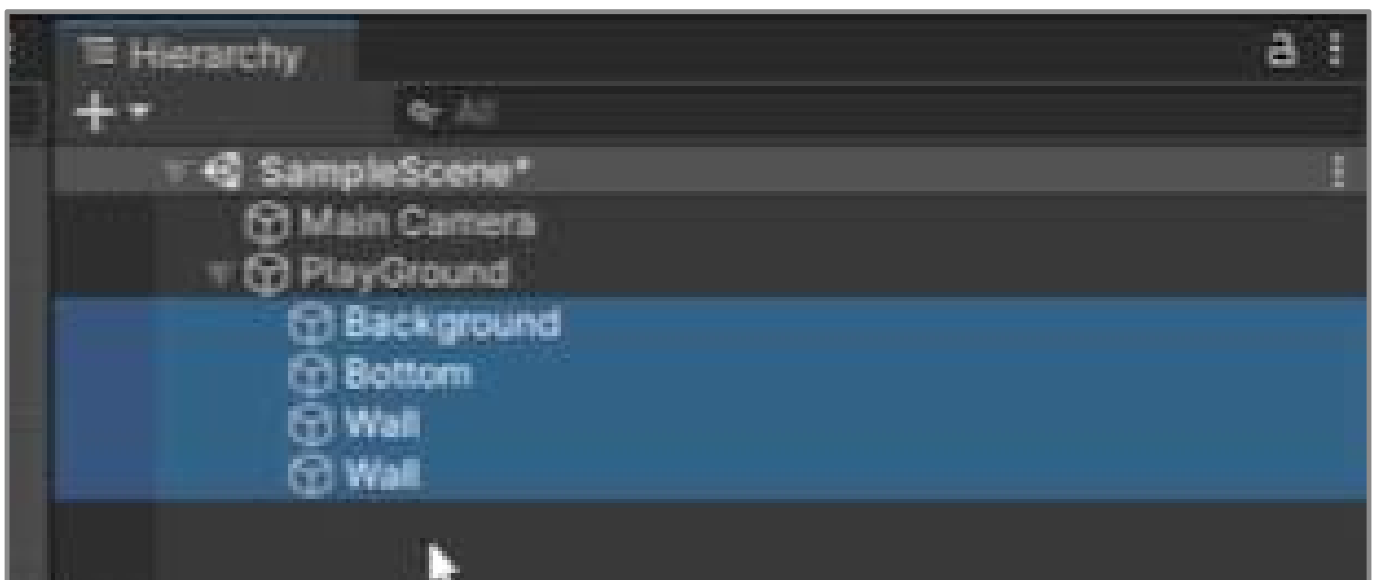


Playground ④ 오브젝트 그룹화

지금까지 배경·바닥·Wall 2개, 총 4개 오브젝트 완성!

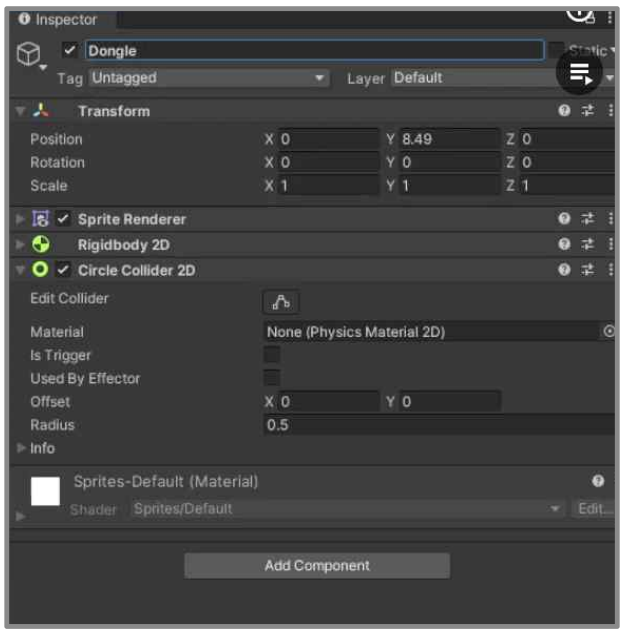
Create Empty로 'Playground' 생성, 위치 (0,0)

4개 오브젝트 다중 선택 후 드래그 → 그룹화 완료!



Dongle(공) 만들고 물리 테스트

2D Object → Sprite → Circle 생성
Rigidbody2D(Dynamic) + Circle Collider2D
Play를 눌러 자유낙하 테스트!



스크립트 생성하기

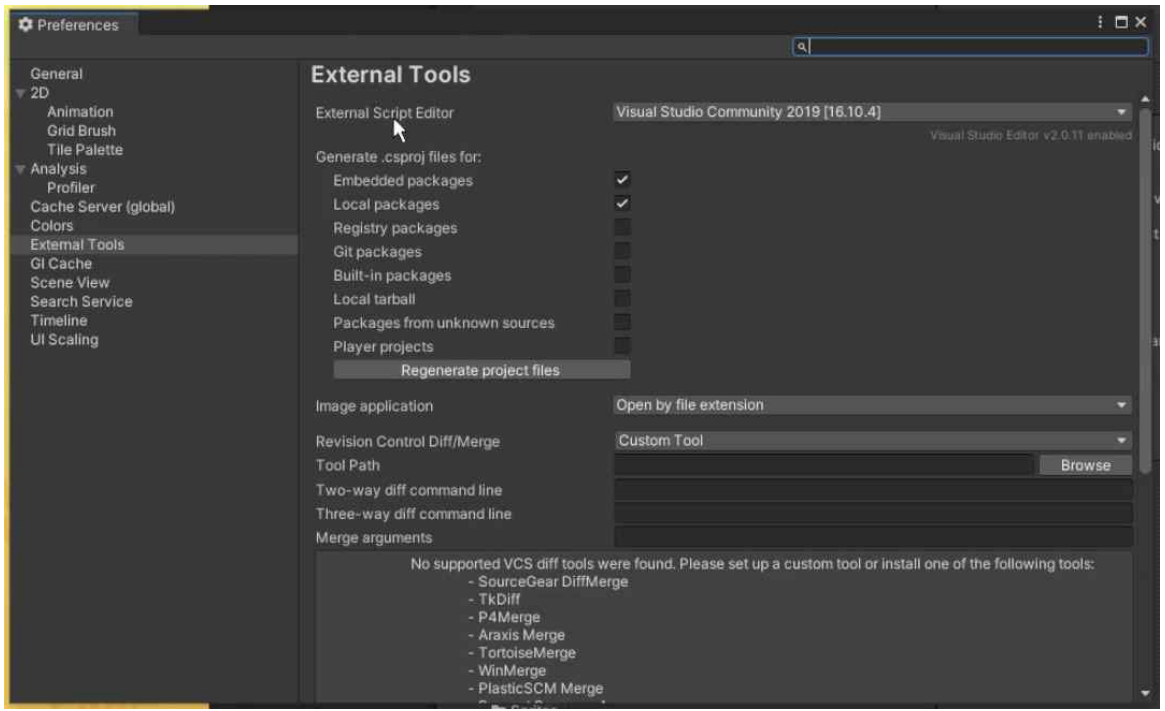
공 오브젝트 이름을 'Dongle'로 변경
Assets 폴더에 'Scripts' 폴더 생성 → C# Script 생성
3~5초 기다려 아이콘이 깜빡이면 생성 완료!
더블클릭 시 Visual Studio가 열립니다



스크립트 에디터 연결 확인

Start / Update / class 등 색깔이 흰색 그대로면 연결이 안 된 것!

Edit -> Preferences->
External Script Editor 확인
'파일 형식으로 열기'가 아니라
Visual Studio Community 버전을 정확히 지정해 주세요



오늘의 핵심 로직: 마우스로 Dongle 조작

마우스를 누르면 Dongle이 마우스를 따라가고,
손을 떼면 다시 자유낙하!

Vector3 mousePosition
Input.mousePosition
으로 마우스 위치(스크린 좌표계)를 가져옴

그런데 이 값을 그대로 대입하면 안 됩니다!

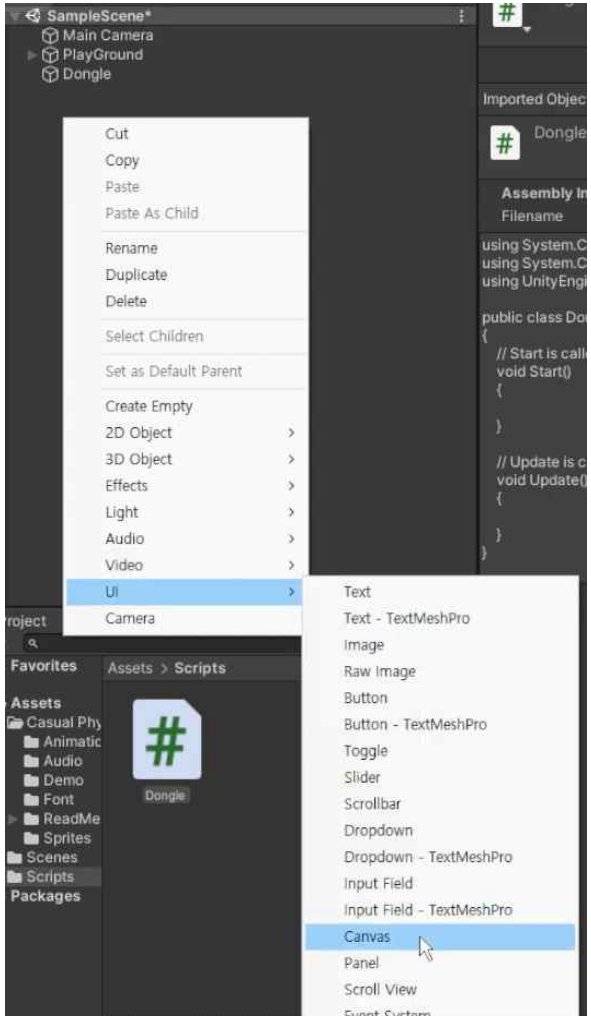
```
1   using System.Collections;
2   using System.Collections.Generic;
3   using UnityEngine;
4
    0 Unity 스크립트 | 참조 0개
5   public class Dongle : MonoBehaviour
6   {
7       // Start is called before the first frame update
        0 Unity 메시지 | 참조 0개
8       void Start()
9       {
10
11      }
12
13      // Update is called once per frame
        0 Unity 메시지 | 참조 0개
14      void Update()
15      {
16          Vector3 mousePos = Input.mousePosition;
17      }
18  }
19
```

왜 안 될까? Screen vs World

Hierarchy 우클릭 → UI → Canvas 생성
(캔버스 = UI를 배치하는 도화지)

Scene에서 캔버스를 더블클릭하면
큰 직육면체가 등장 — 이것이 'Screen'
2D 버튼으로 확인해보면
게임 실행 창과 동일!

'World'는 우리가 실제로
배치한 3D 공간



좌표 변환: ScreenToWorldPoint()

Camera.main으로 메인 카메라 접근 → ScreenToWorldPoint() 함수로 스크린 좌표 → 월드 좌표 변환!

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Unity 스크립트|참조 0개
public class Dongle : MonoBehaviour
{
    // Start is called before the first frame update
    Unity 메시지|참조 0개
    void Start()
    {

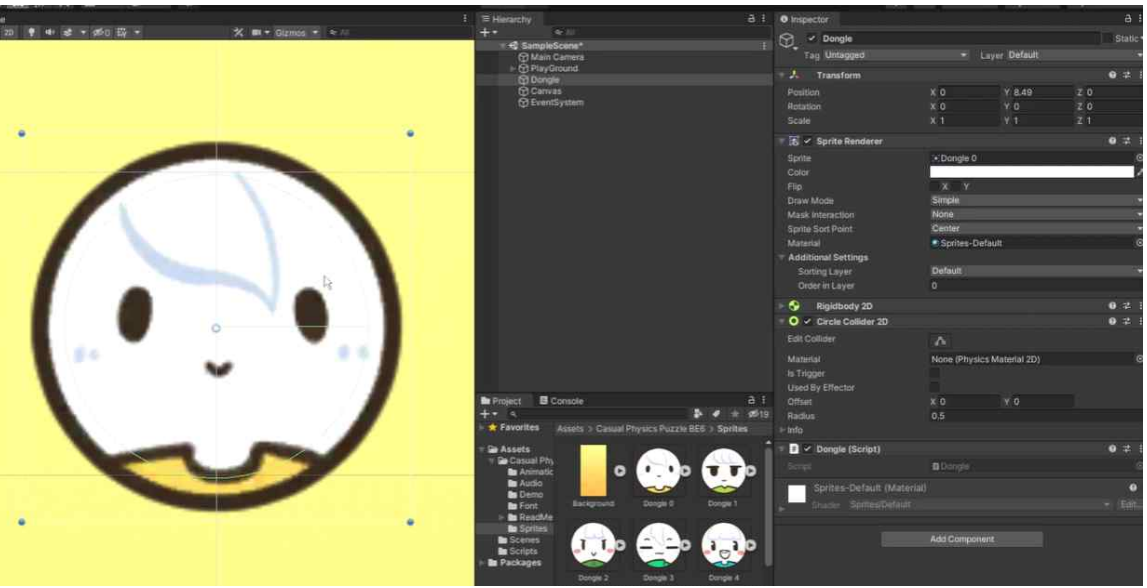
    }

    // Update is called once per frame
    Unity 메시지|참조 0개
    void Update()
    {
        Vector3 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
        transform.position = mousePos;
    }
}
```

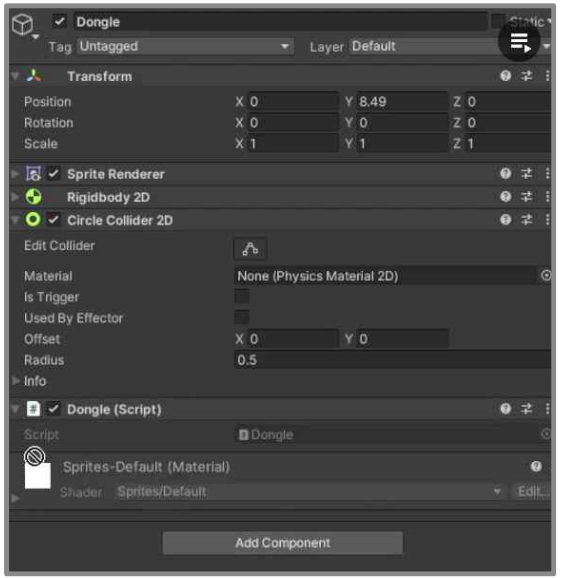
Dongle 모양 다듬기

Transform.position = mousePosition 저장 후,
스크립트를 Dongle에 연결, 스프라이트를 'Dongle' 모양으로 변경
Circle Collider2D 반지름 0.5 → 0.6

① 스프라이트를 'Dongle' 모양으로 변경

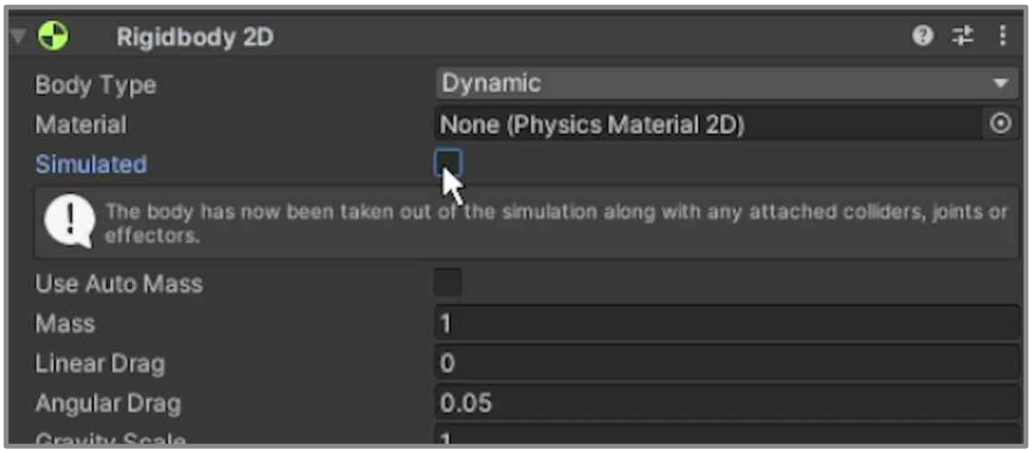


② 스크립트(Dongle) 연결 확인



실행 테스트 & Z축 버그 수정

Rigidbody2D의 Simulated 체크 해제



원인: 카메라 Z가 -10 → mousePosition.z를 0으로 고정

```
14      void Update()
15      {
16          Vector3 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
17          mousePos.z = 0;
18          transform.position = mousePos;
19      }
20  }
```

실행 결과: Dongle이 정상적으로 보임!



부드럽게 따라가기 (Vector3.Lerp)

Vector3.Lerp() 함수 사용
Lerp(현재 위치, 목표 위치, 따라가는 강도)
강도 값은 0~1 사이

0.01로 테스트 → 너무 느림!
0.2로 조정 하면 적당해요

```
14      void Update()
15      {
16          Vector3 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
17          mousePos.z = 0;
18          transform.position = Vector3.Lerp(transform.position, mousePos, 0.01f);
19      }
20  }
21
```

Y축 고정 + 경계선(Border) 만들기

mousePos.y = 8 → Dongle은 위아래로 안 움직임

경계선(Line) 스프라이트를 Playground에 추가
Position Y 6, Is Trigger 체크

색상은 주황색 + 알파값을 낮춰서
배경과 자연스럽게 어울리게



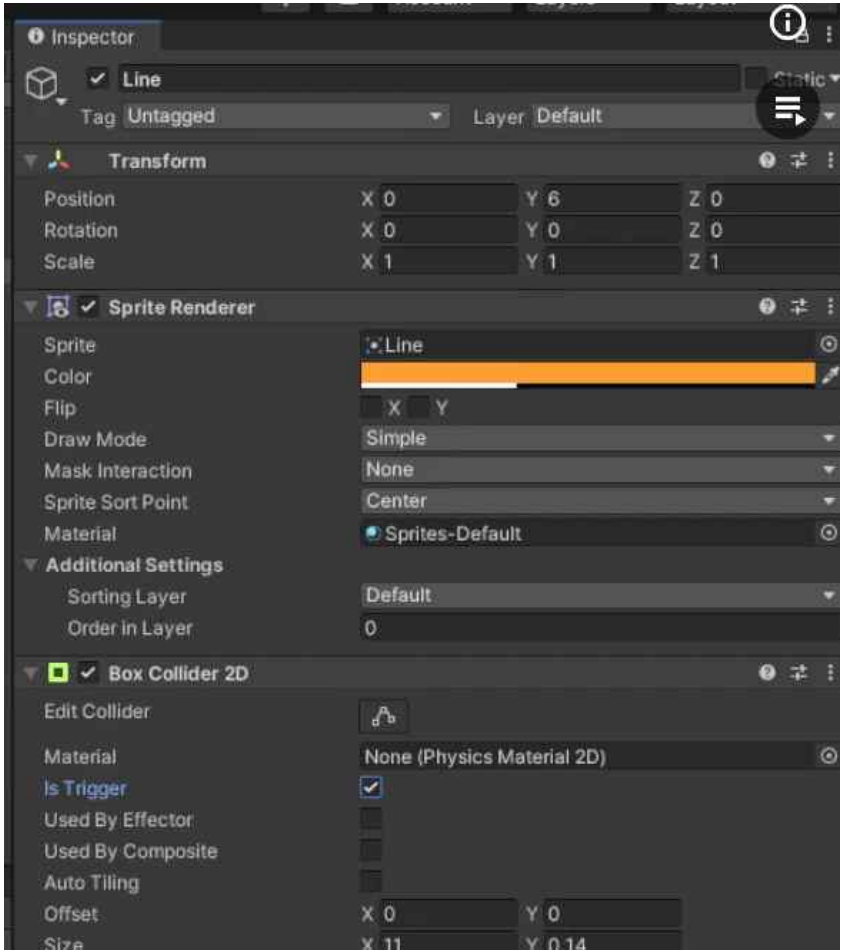


경계선에 Trigger 콜라이더 추가

물리 충돌은 필요 없고
'닿았다'는 표시만 있으면 충분!

Box Collider2D 추가 →
Is Trigger 체크

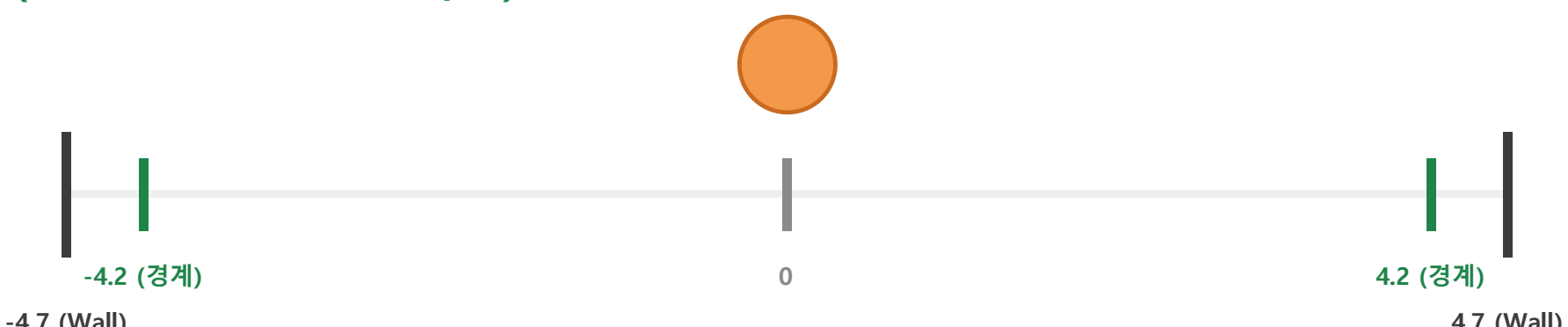
체크 안 하면 Dongle이 경계선
위에서 굴러다니게 됩니다!



좌우 이동 범위 제한하기 ①

왼쪽 Wall 위치 -4.7 + Wall 두께 절반(0.5)
= 왼쪽 경계 -4.2
오른쪽은 반대로 +4.2

Dongle 반지름만큼 여유도 추가 계산
(Transform.localScale / 2)



좌우 이동 범위 제한하기 ②

mousePosition.x가 왼쪽 경계보다 작다? → 왼쪽 경계값으로 고정
mousePosition.x가 오른쪽 경계보다 크다? → 오른쪽 경계값으로 고정
저장 후 실행 → 마우스를 아무리 옆으로 움직여도 Wall 밖으로 안 나갑니다!

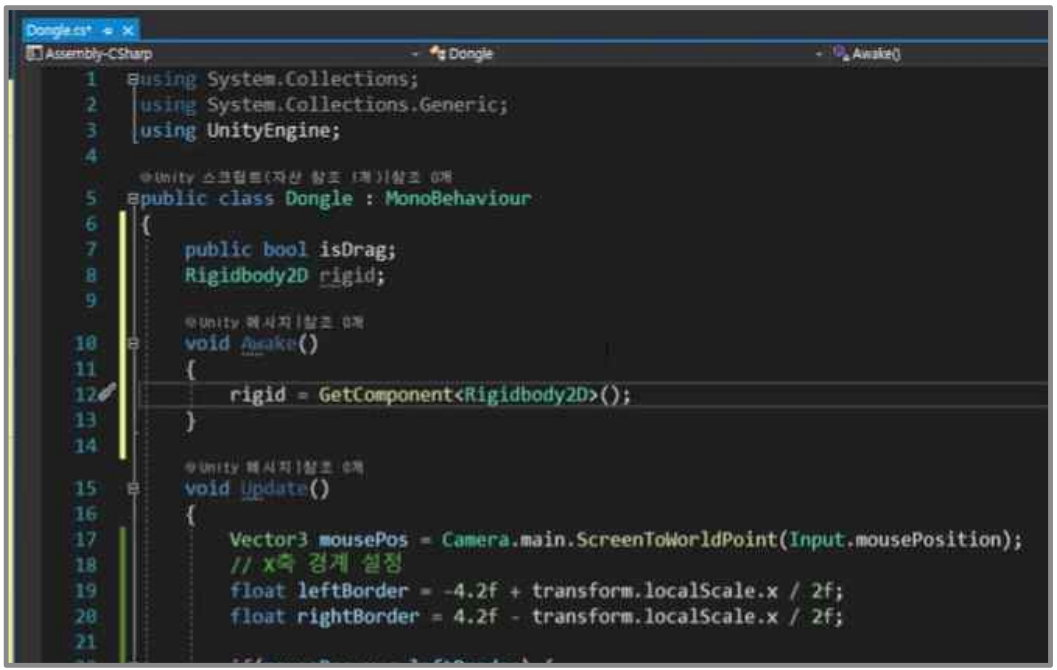
```
void Update()
{
    Vector3 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
    // x축 경계 설정
    float leftBorder = -4.2f + transform.localScale.x / 2f;
    float rightBorder = 4.2f - transform.localScale.x / 2f;

    if(mousePos.x < leftBorder) {
        mousePos.x = leftBorder;
    }
    else if (mousePos.x > rightBorder) {
        mousePos.x = rightBorder;
    }

    mousePos.y = 8;
    mousePos.z = 0;
    transform.position = Vector3.Lerp(transform.position, mousePos, 0.2f);
}
```

눌렀다 뗐다 로직 준비하기

public bool isDrag 변수 선언
Rigidbody2D rigid 변수도 선언
void Awake()에서 rigid = GetComponent<Rigidbody2D>()로 초기화



Drag() / Drop() 함수 만들기

Drag(): isDrag = true (마우스 따라가기 시작)
Drop(): isDrag = false + rigid.simulated = true (자유낙하 시작)
기존 로직은 통째로 Update() 안 if(isDrag) 블록으로 이동

Drag() / Drop() 함수

```
        mousePos.y = 8;
        mousePos.z = 0;
        transform.position = Vector3.Lerp(transform.position, mousePos, 0.2f);
    }

    참조 0개
    public void Drag()
    {
        isDrag = true;
    }

    참조 0개
    public void Drop()
    {
        isDrag = false;
        rigid.simulated = true;
    }
}
```

if(isDrag)로 감싼 Update()

```
@Unity 메시지|참조 0개
void Update()
{
    if(isDrag) {
        Vector3 mousePos = Camera.main.ScreenToWorldPoint(Input.mousePosition);
        // x축 경계 설정
        float leftBorder = -4.2f + transform.localScale.x / 2f;
        float rightBorder = 4.2f - transform.localScale.x / 2f;

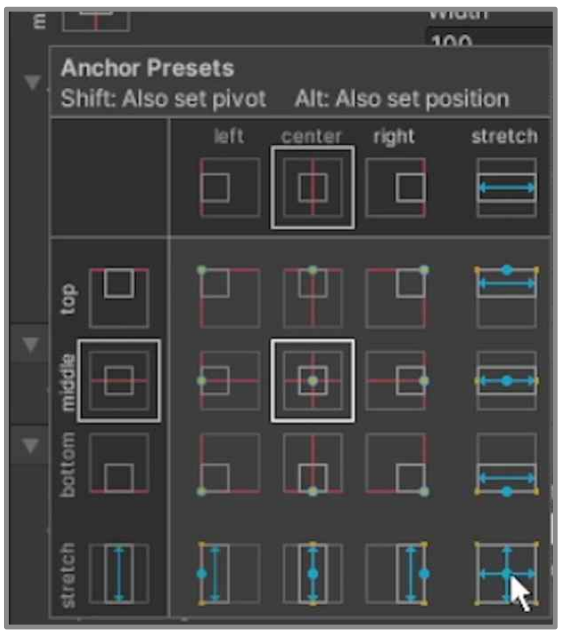
        if (mousePos.x < leftBorder) {
            mousePos.x = leftBorder;
        }
        else if (mousePos.x > rightBorder) {
            mousePos.x = rightBorder;
        }

        mousePos.y = 8;
        mousePos.z = 0;
        transform.position = Vector3.Lerp(transform.position, mousePos, 0.2f);
    }
}
```

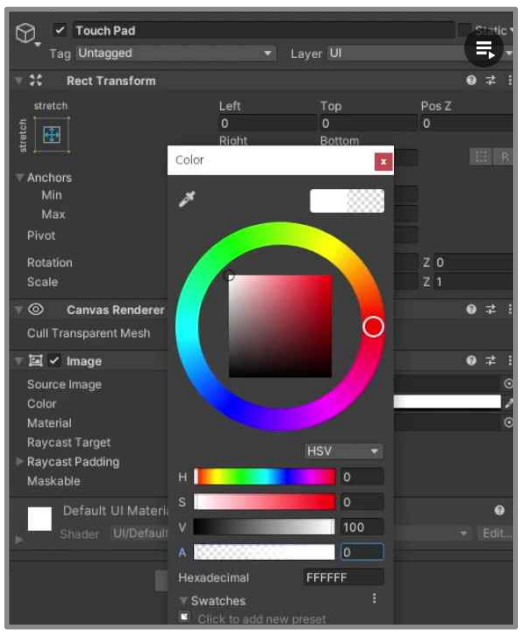
UI로 마우스 입력 받기 (Touch Pad)

Hierarchy 우클릭 → UI → Image 생성, 이름: 'Touch Pad'
앵커에서 Shift+Alt 클릭 → 캔버스 전체 크기로
Color의 Alpha를 0으로 낮춰서 완전히 투명하게!

앵커 Stretch/Stretch 프리셋



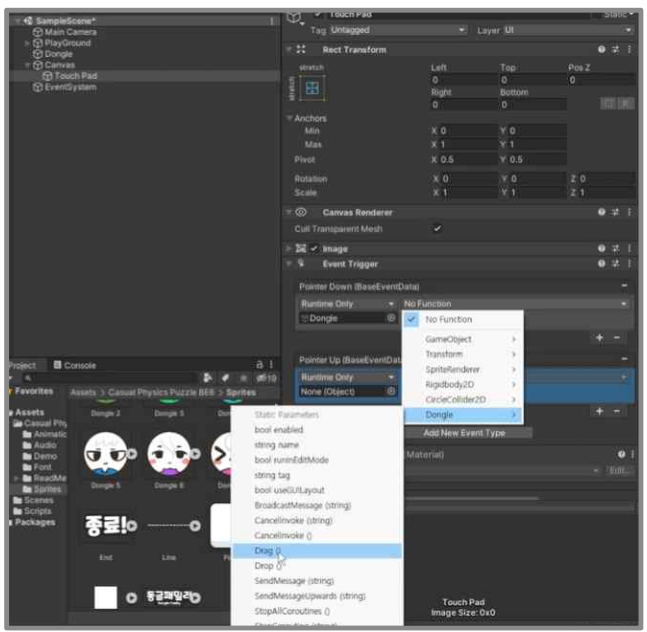
Alpha = 0으로 설정



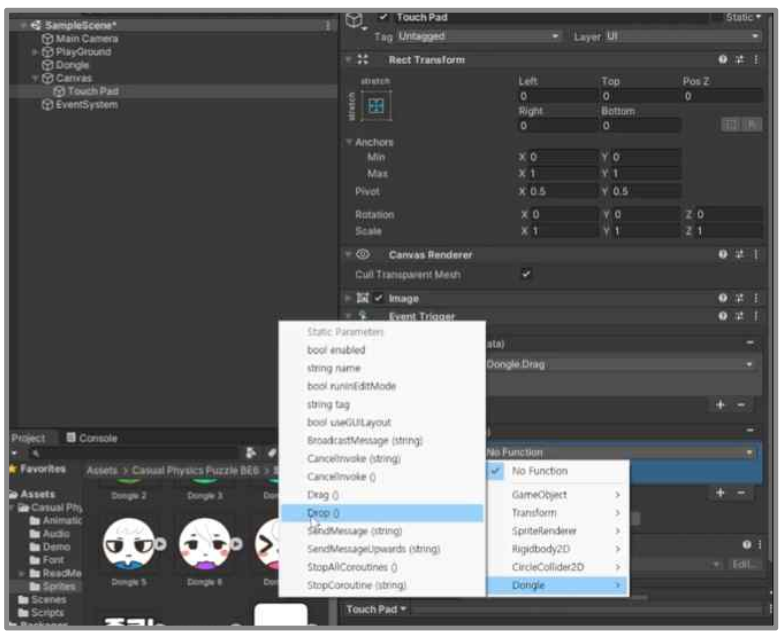
Event Trigger로 클릭 연결하기

Add Component → Event Trigger 추가
Add New Event Type → Pointer Down, Pointer Up 각각 추가
Dongle 오브젝트를 연결하고 각각 Drag() / Drop() 함수를 선택!

Pointer Down → Dongle.Drag()



Pointer Up → Dongle.Drop()



완성! 테스트해보기

마우스를 누르면 Dongle이 따라오고
손을 떼면 자유낙하하는 모습 확인!

Rigidbody2D의 Simulated
가 스크립트로 켜지면서
자유낙하가 시작되는 원리입니다
+

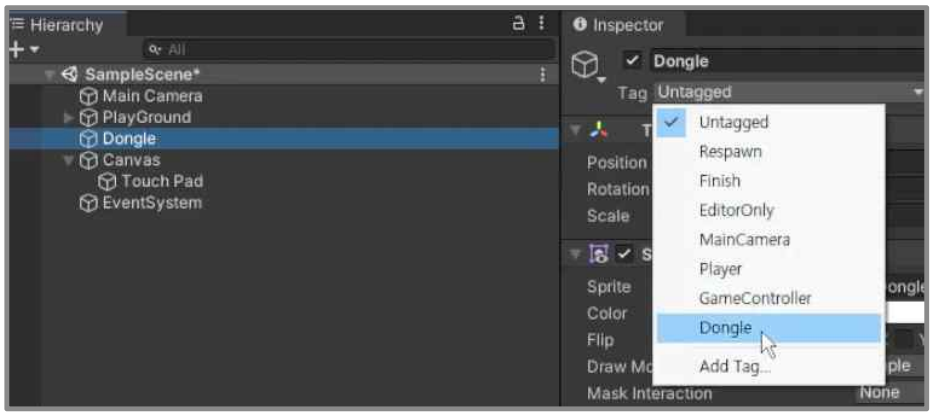
```
public void Drag()
    {
        isDrag = true;
        rigid.simulated = false;
    }
```

필요

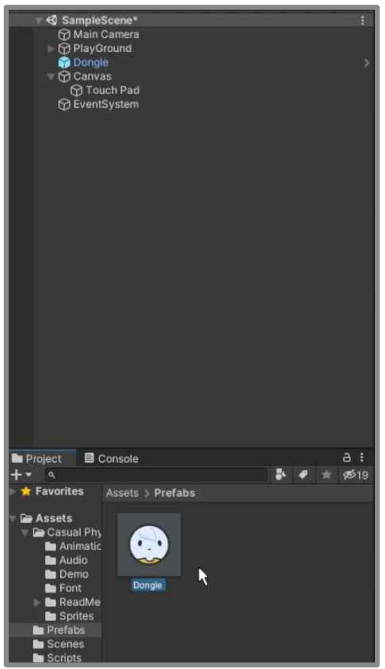
태그 달기 & Prefab으로 저장

Dongle 선택 → Tag에 'Dongle' 새로 추가 후 지정
Assets 폴더에 'Prefabs' 폴더 생성
Dongle 오브젝트를 그 안으로 드래그 → 저장 완료!

Tag 'Dongle' 지정 완료



Prefabs에 저장된 Dongle



다음 시간에는

Playground 제작 +
마우스로 Dongle 조작하는
기본 로직 완성!

다음 시간엔 여러 다른 모양의
Dongle들을 계속 생성해 볼 거예요

다음 시간에 이어서 진행할게요 :)